**Problem.** Build, entirely in basic Python and without using any agent framework or large-language-model API, a simple autonomous "scientist agent" whose task is to identify an unknown mathematical function by interacting with an environment through a limited set of tools. At the start of each run, the environment secretly chooses one function from a small family, for example a linear function $y = ax + b$ or a quadratic function $y = ax^2 + bx + c$, with coefficients hidden from the agent; in a later version, observations may contain noise so that the environment returns $y = f(x) + \varepsilon$, where $\varepsilon$ is a small random error. The agent is given a fixed experimental budget $B$ and must decide sequentially what to do. Its principal action is to choose a value $x_t$ and call an `experiment(`$x_t$`)` tool, after which it receives the observation $y_t = f(x_t)$, or its noisy version, and stores the pair $(x_t, y_t)$ in memory $M_t = \{(x_1, y_1), \ldots, (x_t, y_t)\}$. Using only this accumulated memory, the agent must maintain one or more hypotheses about the hidden function, estimate their parameters from first principles, decide which hypothesis currently explains the observations best, choose the next experiment in a purposeful way, and eventually decide whether to continue gathering information or to stop and return its final model. At minimum, the system should explicitly contain an environment state $S_t$, an observation $O_t$, a set of possible actions $A_t$, a memory $M_t$, a decision policy $\pi$ mapping the available information to the next action, a small collection of tools such as `experiment`, `fit_linear`, `fit_quadratic`, and `validate`, and a stopping rule $\tau$ that either terminates when confidence is sufficiently high or when the budget is exhausted. The fitting routines should be implemented directly using elementary Python arithmetic and lists rather than machine-learning libraries; for instance, students may derive the least-squares estimates for the linear model and solve the corresponding small system for the quadratic model. The quality of the agent should be measured on previously unseen test points using a loss such as mean squared prediction error, while also charging for experimentation, for example through an objective of the form $J = \mathrm{MSE}_{\mathrm{test}} + \lambda N_{\mathrm{queries}}$, so that an agent that obtains good predictions with fewer experiments is preferred. The important part of the task is not merely to fit a curve after receiving data, but to implement the complete closed loop observe $\to$ update memory $\to$ form or revise hypothesis $\to$ choose action $\to$ observe again $\to$ stop and to explain mathematically why each component is needed. Begin with noiseless linear functions, then extend the same code to distinguish between linear and quadratic functions, then introduce noisy observations, and finally design a policy that chooses informative values of $x$ and decides by itself when enough evidence has been collected to stop.